

Fitting the outcome model

Malcolm Barrett
Stanford University

Outcome Model

```
1 library(broom)
2
3 lm(outcome ~ exposure, data = df, weights = wts) |>
4   tidy()
```



This will get us the point estimate



This will NOT get us the correct confidence intervals



There are three other approaches we will consider

Three ways to estimate the uncertainty

- 1 **A bootstrap**: correct and general purpose, but slow
- 2 **A robust standard error**: a sandwich estimator for the outcome model alone
- 3 **A propensity-aware sandwich**: correct and fast

The bootstrap is still worth knowing: it also solves problems that have no closed-form solution

1. Create a function to run your analysis once on a sample of your data

```
1 fit_ipw <- function(.split, ...) {  
2   .df <- as.data.frame(.split)  
3  
4   # fit propensity score model  
5   propensity_model <- glm(  
6     net ~ income + health + temperature,  
7     family = binomial(),  
8     data = .df  
9   )  
10  
11  # calculate inverse probability weights  
12  .df <- propensity_model |>  
13    augment(type.predict = "response", data = .df) |>  
14    mutate(wts = wt_ate(.fitted, net, exposure_type = "binary"))  
15  
16  # fit correctly bootstrapped ipw model  
17  lm(malaria_risk ~ net, data = .df, weights = wts) |>  
18    tidy()  
19 }
```

2. Use {rsample} to bootstrap our causal effect

```
1 library(rsample)
2
3 # fit ipw model to bootstrapped samples
4 bootstrapped_net_data <- bootstraps(
5   net_data,
6   times = 1000,
7   apparent = TRUE
8 )
9
10 bootstrapped_net_data
```

2. Use {rsample} to bootstrap our causal effect

```
# Bootstrap sampling with apparent sample
```

```
# A tibble: 1,001 × 2
```

	splits	id
	<list>	<chr>
1	<split [1752/640]>	Bootstrap0001
2	<split [1752/638]>	Bootstrap0002
3	<split [1752/637]>	Bootstrap0003
4	<split [1752/635]>	Bootstrap0004
5	<split [1752/639]>	Bootstrap0005
6	<split [1752/616]>	Bootstrap0006
7	<split [1752/652]>	Bootstrap0007
8	<split [1752/638]>	Bootstrap0008
9	<split [1752/629]>	Bootstrap0009
10	<split [1752/641]>	Bootstrap0010

2. Use {rsample} to bootstrap our causal effect

```
1 fit_ipw(bootstrapped_net_data$splits[[1]])
```

```
# A tibble: 2 × 5
```

	term	estimate	std.error	statistic	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	(Intercept)	43.4	0.470	92.3	0
2	netTRUE	-12.3	0.663	-18.5	8.96e-70

2. Use {rsample} to bootstrap our causal effect

```
1 ipw_results <- bootstrapped_net_data |>
2   mutate(boot_fits = map(splits, fit_ipw))
3
4 ipw_results
```

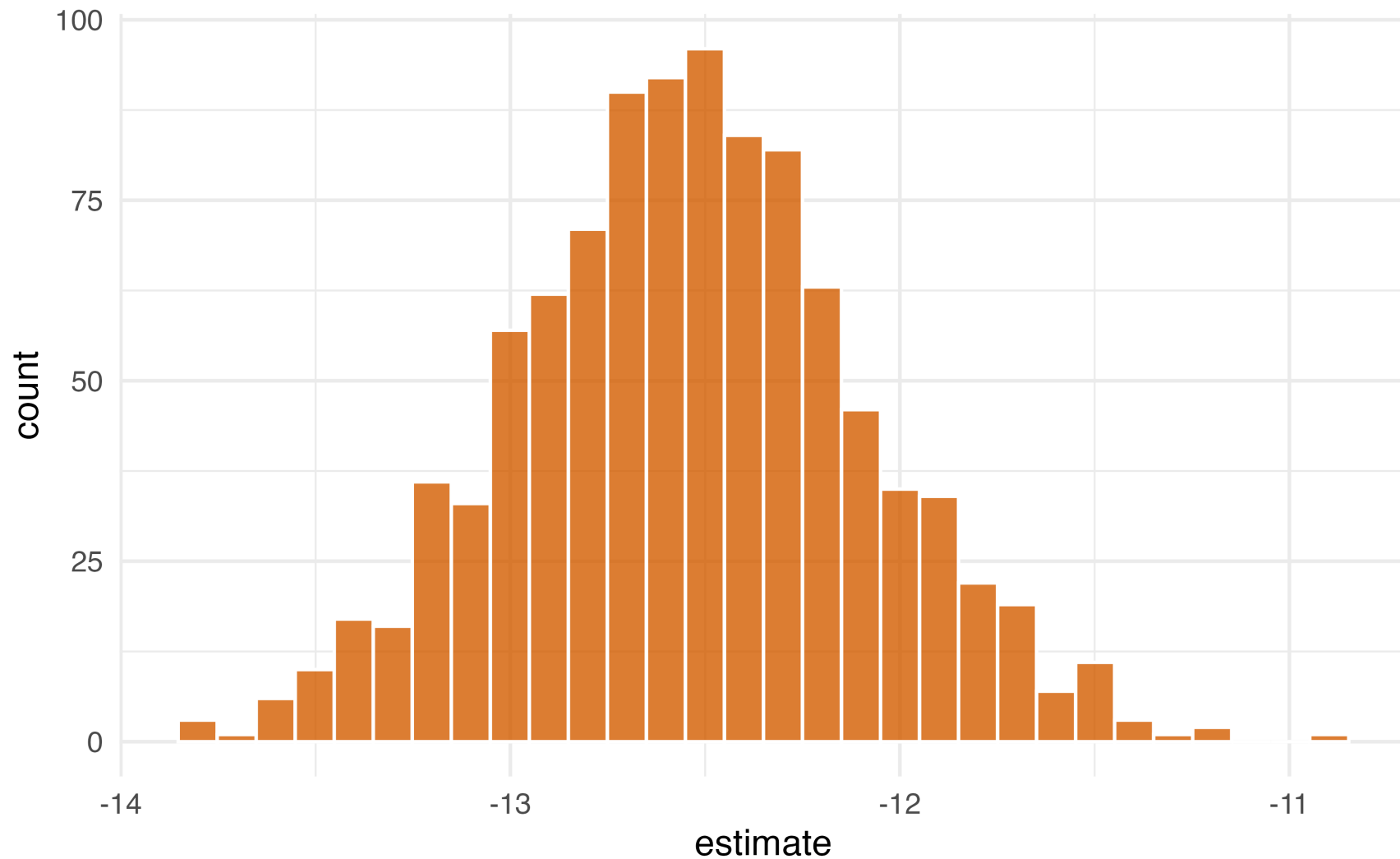
2. Use {rsample} to bootstrap our causal effect

```
# Bootstrap sampling with apparent sample
```

```
# A tibble: 1,001 × 3
```

	splits	id	boot_fits
	<list>	<chr>	<list>
1	<split [1752/640]>	Bootstrap0001	<tibble [2 × 5]>
2	<split [1752/638]>	Bootstrap0002	<tibble [2 × 5]>
3	<split [1752/637]>	Bootstrap0003	<tibble [2 × 5]>
4	<split [1752/635]>	Bootstrap0004	<tibble [2 × 5]>
5	<split [1752/639]>	Bootstrap0005	<tibble [2 × 5]>
6	<split [1752/616]>	Bootstrap0006	<tibble [2 × 5]>
7	<split [1752/652]>	Bootstrap0007	<tibble [2 × 5]>
8	<split [1752/638]>	Bootstrap0008	<tibble [2 × 5]>
9	<split [1752/629]>	Bootstrap0009	<tibble [2 × 5]>
10	<split [1752/654]>	Bootstrap0010	<tibble [2 × 5]>

2. Use `{rsample}` to bootstrap our causal effect



3. Pull out the causal effect

```
1 # get t-statistic-based CIs
2 boot_estimate <- int_t(ipw_results, boot_fits) |>
3   filter(term == "netTRUE")
4
5 boot_estimate
```

```
# A tibble: 1 × 6
  term      .lower .estimate .upper .alpha .method
  <chr>    <dbl>    <dbl>  <dbl>  <dbl>  <chr>
1 netTRUE -13.4      -12.5  -11.6   0.05  student-t
```

Fit both models on the full data

```
1 propensity_model <- glm(  
2   net ~ income + health + temperature,  
3   family = binomial(),  
4   data = net_data  
5 )  
6  
7 net_data_wts <- propensity_model |>  
8   augment(type.predict = "response", data = net_data) |>  
9   mutate(wts = wt_ate(.fitted, net, exposure_type = "binary"))  
10  
11 ipw_model <- lm(malaria_risk ~ net, data = net_data_wts, weights = wts)
```

Robust standard errors

The sandwich estimator gets its name from the form of the variance it calculates: a “meat” matrix built from the variability of each observation’s residuals, sandwiched between two copies of a “bread” matrix that comes from the design of the model

When the model’s assumptions hold, the bread and meat combine to give the usual model-based variance

When they do not, as with IPW weights, the sandwich form still gives a consistent variance estimate

That is why these are called “robust” standard errors

Robust standard errors

```
1 vcov(ipw_model)
```

```
              (Intercept)      netTRUE  
(Intercept)    0.1953735 -0.1953735  
netTRUE        -0.1953735  0.3896912
```

`vcov()` returns the variance-covariance matrix that `tidy()` and `summary()` use under the hood; it treats the weights as fixed, which they are not

Robust standard errors

```
1 library(sandwich)
2
3 sandwich(ipw_model)
```

```
              (Intercept)      netTRUE
(Intercept)    0.157519 -0.1575190
netTRUE        -0.157519  0.5720561
```

For the treatment coefficient, the sandwich variance is about 1.5 times the model-based one, so the model-based SE understates the uncertainty; for the intercept, it is smaller

Robust standard errors

```
1 robust_se <- avg_comparisons(ipw_model, variables = "net", vcov = "HC3")
2
3 robust_se
```

Estimate	Std. Error	z	Pr(> z)	S	2.5 %	97.5 %
-12.5	0.758	-16.5	<0.001	201.7	-14	-11.1

Term: net

Type: response

Comparison: TRUE - FALSE

Note

`avg_comparisons()` returns a marginal effect: the average of the predicted outcome difference across the observed covariate distribution. `vcov = "HC3"` requests the HC3 variant of the sandwich. We will see more of `marginalEffects` when we get to g-computation.

 Robust SEs account for the weights, but not for estimating them

ipw(): correct and fast

- `ipw()` uses a sandwich estimator that also accounts for the uncertainty in the propensity score model
- Bring your own models: you fit the propensity score and outcome models yourself, then supply them to `ipw()`

```
1 ipw_fit <- ipw(propensity_model, ipw_model)
```

ipw(): correct and fast

```
1 ipw_fit
```

Inverse Probability Weight Estimator

Estimand: ATE

Effects: marginal (population-averaged)

Weight Estimator:

```
Call: glm(formula = net ~ income + health + temperature, family = binomial(),
  data = net_data)
```

Outcome Model:

```
Call: lm(formula = malaria_risk ~ net, data = net_data_wts, weights = wts)
```

Marginal estimates:

	estimate	std.err	z	ci.lower	ci.upper	conf.level	p.value
diff	-12.54476	0.43712	-28.698	-13.402	-11.688	0.95	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

ipw() reports the estimand along with the estimate

Comparing the three approaches

```
# A tibble: 3 × 4
  method      estimate conf.low conf.high
  <chr>         <dbl>    <dbl>    <dbl>
1 bootstrap   -12.5    -13.4    -11.6
2 HC3 robust SE -12.5    -14.0    -11.1
3 ipw()       -12.5    -13.4    -11.7
```

`ipw()` fits each model once and takes a fraction of a second; the
`bootstrap` fits 1,001 pairs of models

Your Turn

Create a function called `fit_ipw` that fits the propensity score model and the weighted outcome model for the effect between `park_extra_magic_morning` and `wait_minutes_posted_avg`

Using the `bootstraps()` and `int_t()` functions to estimate the final effect.

Fitting both models on the full data and using `ipw()` to compare the two intervals.

Binary outcomes

```
1 net_data_wts <- net_data_wts |>
2   mutate(risk_over_50 = malaria_risk > 50)
3
4 binary_outcome_model <- glm(
5   risk_over_50 ~ net,
6   data = net_data_wts,
7   weights = wts,
8   # quasibinomial() avoids a warning about non-integer successes
9   family = quasibinomial()
10 )
```

Binary outcomes: all three effects

```
1 ipw(propensity_model, binary_outcome_model) |>  
2   tidy(conf.int = TRUE, exponentiate = TRUE) |>  
3   select(term, estimate, conf.low, conf.high)
```

```
# A tibble: 3 × 4  
  term      estimate conf.low conf.high  
  <chr>      <dbl>    <dbl>    <dbl>  
1 rd        -0.205    -0.240    -0.169  
2 rr         0.289     0.202     0.415  
3 or         0.225     0.152     0.334
```

`ipw()` reports the risk difference, risk ratio, and odds ratio, with standard errors that account for the propensity score model